

### Problem 1

Consider a network of  $N \in \mathbb{N}$  agents and the interaction between them is modeled via a graph  $\mathcal{G}(\mathcal{V}, \mathcal{E})$ . Let the corresponding row-stochastic adjacency matrix be denoted by  $A \in \mathbb{R}^{N \times N}$ . Let  $v \in \mathbb{R}_{\geq 0}^N$  be the normalized left dominant eigenvector such that  $v^\top A = v^\top$  and  $\mathbf{1}_N^\top v = 1$ . For the consensus system  $x[k+1] = Ax[k]$ , the disagreement vector  $\delta[k] \in \mathbb{R}^N$  at time  $k \in \mathbb{N}$  is

$$\delta[k] = x[k] - (v^\top x[k])\mathbf{1}_N.$$

Note that  $v^\top x[k]$  corresponds to the normalized weighted average of the entries of  $x[k]$  at time  $k$ .

1. Prove that  $v^\top x[k]$  is constant over all time steps  $k \in \mathbb{N}$ .
2. Show that  $(A - \mathbf{1}_N v^\top) = (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top)$ .
3. Show that the dynamics of the disagreement vector is given by

$$\delta[k+1] = (A - \mathbf{1}_N v^\top)\delta[k].$$

*Solution: 1:*

Since,  $v$  is a fixed normalized left eigenvector, for  $v^\top x[k]$  to be constant over all time steps ( $k \in \mathbb{N}$ ), it should satisfy

$$v^\top x[k+1] = v^\top x[k] \quad \forall k \in \mathbb{N}$$

and using consensus dynamics,  $x[k+1] = Ax[k]$ , LHS equals:

$$\begin{aligned} &\Rightarrow v^\top (Ax[k]) \\ &\Rightarrow (v^\top A)x[k] \quad (\because \text{via associative property}) \end{aligned}$$

since  $v$  is left eigenvector of the matrix  $A$ ,

$$\begin{aligned} v^\top A &= v^\top \\ &\Rightarrow v^\top x[k] \quad \text{which equals RHS} \end{aligned}$$

Thus,  $v^\top x[k+1] = v^\top x[k]$  holds for every step and by induction,

$$v^\top x[k] = v^\top x[0] = \text{constant}, \quad \forall k \in \mathbb{N} \quad (\text{where } x[0] \text{ denotes the initial condition})$$

which means the weighted average of the agents' states ( $v^\top x[k]$ ) is fixed from the initial condition.

**2:**

To show that,

$$(A - \mathbf{1}_N v^\top) = (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top)$$

Expanding the RHS:

$$\begin{aligned} \text{RHS} &= (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top) \\ &= AI_N - A(\mathbf{1}_N v^\top) - \mathbf{1}_N I_N v^\top + \mathbf{1}_N v^\top \mathbf{1}_N v^\top \\ &= A - (A\mathbf{1}_N)v^\top - \mathbf{1}_N v^\top + \mathbf{1}_N(v^\top \mathbf{1}_N)v^\top && (\because v \text{ is normalized, so } v^\top \mathbf{1}_N = 1) \\ &= A - \mathbf{1}_N v^\top - \mathbf{1}_N v^\top + \mathbf{1}_N v^\top && (\because A \text{ is row-stochastic, so } A\mathbf{1}_N = \mathbf{1}_N) \\ &= A - \mathbf{1}_N v^\top = \text{LHS} \end{aligned}$$

*interpretation :*

- The operator  $I - \mathbf{1}_N v^\top$  removes the weighted average from a state, giving the disagreement at the current time.
- The operator  $A - \mathbf{1}_N v^\top$  applies the dynamics  $A$  and then removes the weighted average, giving the disagreement at the next time step.
- The identity holds because:

$$v^\top (A - \mathbf{1}_N v^\top) = v^\top A - v^\top \mathbf{1}_N v^\top = v^\top - v^\top = 0.$$

Thus, for any  $x$ , the vector  $y = (A - \mathbf{1}_N v^\top)x$  satisfies  $v^\top y = 0$ . This means  $y$  has zero weighted average, i.e., it lies in the disagreement subspace. And hence, applying the projection  $I - \mathbf{1}_N v^\top$  to  $y$  leaves it unchanged.

**3:**

Required: To show that

$$\delta[k+1] = (A - \mathbf{1}_N v^\top) \delta[k]$$

From the definition of the disagreement vector,  $\delta[k] = x[k] - (v^\top x[k]) \mathbf{1}_N$

$$\implies \delta[k+1] = x[k+1] - (v^\top x[k+1]) \mathbf{1}_N \quad (\text{disagreement vector at time step } k+1)$$

From Part 1, we know that  $v^\top x[k+1] = v^\top x[k]$ :

$$\implies \delta[k+1] = x[k+1] - (v^\top x[k]) \mathbf{1}_N$$

And using the consensus dynamics,  $x[k+1] = Ax[k]$ :

$$\begin{aligned} \implies \delta[k+1] &= Ax[k] - (v^\top x[k]) \mathbf{1}_N \\ &= Ax[k] - \mathbf{1}_N v^\top x[k] \\ &= (A - \mathbf{1}_N v^\top) x[k] \end{aligned} \tag{1}$$

So, consider multiplying  $\delta[k]$  by  $(A - \mathbf{1}_N v^\top)$ , which we are yet to show:

$$\begin{aligned} (A - \mathbf{1}_N v^\top) \delta[k] &= (A - \mathbf{1}_N v^\top) (I_N - \mathbf{1}_N v^\top) x[k] && (\text{from the definition of } \delta[k]) \\ &= (A - \mathbf{1}_N v^\top) x[k] && (\text{using the identity from Part 2}) \end{aligned}$$

Comparing this with equation (1), we see that:

$$\delta[k+1] = (A - \mathbf{1}_N v^\top) x[k] = (A - \mathbf{1}_N v^\top) \delta[k]$$

Hence,  $\delta[k+1] = (A - \mathbf{1}_N v^\top) \delta[k]$ .

---

### Problem 2 Solution:

#### 1: W-MSR simulation ( $F = 1$ local adversary)

In W-MSR, each benign node (innocent) collects its neighbours' states plus its own, sorts them, discards up to  $F$  values strictly larger and up to  $F$  strictly smaller than its own state, and updates to the average of the remaining ones. And, the malicious node is free to send arbitrary values; here it sends a large oscillating signal.

#### Python Code

```

import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

G = nx.Graph()
edges = [
    (1, 5), (1, 6), (1, 7),
    (2, 5), (2, 6), (2, 8), (2, 4),
    (3, 5), (3, 7), (3, 8), (3, 4),
    (5, 6), (6, 8), (8, 7), (7, 5),
    (5, 8), (6, 7), (8, 4),
]
G.add_edges_from(edges)

N, F, T = 8, 1, 25
np.random.seed(42)

x = np.zeros((N, T))
x[:, 0] = np.random.uniform(10, 50, 7)

# node 8 is malicious
mal = 7

# malicious node's state
for k in range(T):
    x[mal, k] = 80 + 20 * np.sin(k)

# W-MSR updates
for k in range(T - 1):

    x_next = x[:, k].copy()

    for i in range(N):

        # malicious node does not follow W-MSR
        if i == mal:
            continue

        # neighbors + node itself
        nbrs = [j - 1 for j in G.neighbors(i + 1)] + [i]

        # store (node, value)
        values = [(j, x[j, k]) for j in nbrs]

        xi = x[i, k]

        # values strictly smaller than x_i (sorted ascending to discard smallest)
        lower = sorted([(j, v) for j, v in values if v < xi], key=lambda p: p[1])

        # values strictly larger than x_i (sorted descending to discard largest)
        upper = sorted(
            [(j, v) for j, v in values if v > xi], key=lambda p: p[1], reverse=True
        )

        # discard up to F values on each side of x_i
        discard = set()

        for j, _ in lower[:F]:
            discard.add(j)

```

```

for j, _ in upper[:F]:
    discard.add(j)

# remaining values
kept = [v for j, v in values if j not in discard]

if len(kept) == 0:
    kept = [xi]

x_next[i] = np.mean(kept)

# store all benign updates at once
x[:, k + 1] = x_next

# keep malicious node's predefined value
x[mal, k + 1] = 80 + 20 * np.sin(k + 1)

plt.figure(figsize=(8, 4.5))

for i in range(N):
    ls = 'r--' if i == mal else '-'
    plt.plot(
        x[i, :], ls, label=f'Node {i+1}' + (' (Mal)' if i == mal else '')
    )

plt.xlabel('Time step $k$')
plt.ylabel('State $x_i[k]$')
plt.title('W-MSR resilient consensus ($F=1$)')
plt.legend(bbox_to_anchor=(1.04, 1), loc='upper left')
plt.grid(True, linestyle=':', alpha=0.7)
plt.tight_layout()
plt.savefig('wmsr_.png')
plt.close()

```

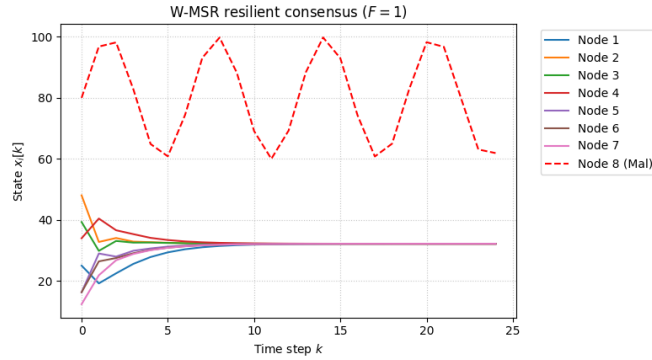


Figure 1: consensus

**conclusion:** The plot shows that W-MSR protocol prevents the malicious node from drifting the benign nodes to an arbitrary value. At every iteration, each benign node removes up to  $F$  neighbor values that are strictly smaller than its own state and up to  $F$  values that are strictly larger than its own state, and then updates using the remaining values. Since the graph is 3-robust and  $F = 1$ , the benign agents converge to a common value despite the malicious behavior.

## 2: Proving the graph is NOT (3, 2)-robust

A graph is  $(r, s)$ -robust if for any two disjoint, non-empty node sets  $S_1, S_2 \subset \mathcal{V}$ , at least one set is  $r$ -accessible (meaning it contains at least  $\min(s, |S_k|)$  nodes, here  $k \in \{1, 2\}$ , that each have  $\geq r$  neighbors in the other set).

to disprove (3, 2)-robustness ( $r = 3, s = 2$ ), we just need to find one partition where both sets fail the accessibility requirement.

So, let's assume:

$$S_1 = \{1, 5\}, \quad S_2 = \{2, 3, 4, 6, 7, 8\}.$$

for each subset:

- $S_1 = \{1, 5\}$  ( $|S_1| = 2$ ): We need  $\min(2, 2) = 2$  nodes in  $S_1$  with  $\geq 3$  neighbors in  $S_2$ .
  - Node 1 connects to  $\{5, 6, 7\}$ . Its neighbors in  $S_2$  are  $\{6, 7\}$ , which is only 2 external neighbors ( $< 3$ ).
  - Node 5 connects to  $\{1, 2, 3, 6, 7, 8\}$ . Its neighbors in  $S_2$  are  $\{2, 3, 6, 7, 8\}$ , which gives 5 external neighbors ( $\geq 3$ ).

Only 1 node (Node 5) in  $S_1$  has  $\geq 3$  external neighbors. Since we needed 2,  $S_1$  is not 3-accessible.

- $S_2 = \{2, 3, 4, 6, 7, 8\}$  ( $|S_2| = 6$ ): We need  $\min(2, 6) = 2$  nodes in  $S_2$  with  $\geq 3$  neighbors in  $S_1$ .
  - Because  $|S_1| = 2$ , no node in the entire graph can have more than 2 neighbors in  $S_1$ .
  - Therefore, zero nodes in  $S_2$  have  $\geq 3$  external neighbors.

So  $S_2$  is not 3-accessible.

Since neither  $S_1$  nor  $S_2$  satisfies the accessibility condition for this partition, the graph is not (3, 2)-robust.

## 3: strategy against a good eavesdropper

Let  $\mathcal{V}_b = \{1, \dots, 7\}$  denote benign nodes and  $\mathcal{N}_8 = \{2, \dots, 7\}$  denote benign neighbors of malicious Node 8.

**Convex Hull constraint & eavesdropper threat:** Under W-MSR, benign updates are convex combinations of retained values, restricting all states to the initial convex hull  $\mathcal{C}_0 = [\min_i x_i[0], \max_i x_i[0]]$ . Unbounded drift to infinity is mathematically impossible. Instead, Node 8 aims to stealthily bias consensus toward the upper boundary  $\max_i x_i[0]$  without triggering the eavesdropper's threshold  $\tau > 0$  or W-MSR rejection.

**stealthy bias strategy:**

- a) **Admissible set intersection ( $K[k]$ ):** Let  $K_i[k] = [v_i^{(F+1)}[k], v_i^{(|\mathcal{N}_i|-F)}[k]]$  be the retained interval at neighbor  $i \in \mathcal{N}_8$ . Node 8 selects its transmission  $z[k]$  within  $K[k] = \bigcap_{i \in \mathcal{N}_8} K_i[k]$ , guaranteeing acceptance by all neighbors ( $K[k] \neq \emptyset$  due to 3-robustness).
- b) **Bounded evasion:** Tracking the local median  $m[k] = \text{median}(\{x_j[k]\}_{j \in \mathcal{N}_8})$ , Node 8 transmits  $z[k] = \min(\sup K[k], m[k] + \delta)$  with  $0 < \delta < \tau$ . Because  $|z[k] - m[k]| \leq \delta < \tau$ , the perturbation remains indistinguishable from noise to the spy while persistently pulling consensus upward.

## 4: Encryption vs. W-MSR & Literature Review

**decrypt-first necessity:** W-MSR relies on non-linear order statistics (sorting and inequality filtering). Standard Homomorphic Encryption (HE) schemes (e.g., Paillier) evaluate additions and multiplications over ciphertexts but cannot perform comparisons without decryption. Order-Preserving Encryption (OPE) permits comparisons but leaks rank distributions to eavesdroppers. Thus, standard setups must **decrypt first** to filter in plaintext.

**Privacy issues of decrypt-first:**

- Raw neighbor states exist in plaintext memory during filtering; compromising one node exposes its entire neighborhood.
- **Overhead:** Decrypting  $O(|\mathcal{N}_i|)$  ciphertexts per step induces latency, while pairwise keys scale as  $O(N^2)$ .

### Literature Review:

*Paper 1 (Huang, Wu & Huang, 2025):* Proposes HE-MSR to resolve plaintext exposure. Agents use additive homomorphism to blind state differences  $(\Delta_{ij} + r_{ij})$  with secret half-weights. The receiver decrypts only masked differences rather than raw states  $x_j[k]$ , executing MSR filtering without memory exposure at the cost of cryptographic computation.

*Paper 2 (Fiore & Russo, 2019):* Proposes DP-MSR to bypass encryption entirely. Agents add calibrated Laplace noise to local states prior to broadcast, allowing benign nodes to run standard W-MSR directly on noisy plaintexts. This guarantees  $(\epsilon, \delta)$ -differential privacy but trades exact consensus for bounded variance around the mean.

**Comparison:** Decrypt-first W-MSR leaks raw states during filtering. Fiore & Russo eliminate encryption by trading accuracy for differential privacy, whereas Huang et al. maintain exact accuracy by encrypting blinded state differences.

## References

- [1] M. Huang, Y. Wu, and Q. Huang, “Resilient and privacy-preserving consensus for multi-agent systems,” *Information Sciences*, vol. 700, p. 121843, 2025.
- [2] D. Fiore and G. Russo, “Resilient consensus for multi-agent systems subject to differential privacy requirements,” *Automatica*, vol. 106, pp. 18–26, 2019.

## AI Declaration

I used deepseek only to help understand the research papers for part 4 of problem 2. All solutions, code, and interpretations are my own.